

MATH2270/MATH2237/MATH2404 Assignment 3

Name

Prathibha Magesh

Student number

S3859590

Assessment declaration checklist

Please carefully read the statements below and check each box if you agree with the declaration. If you do not check all boxes, your assignment will not be marked. If you make a false declaration on any of these points, you may be investigated for academic misconduct. Students found to have breached academic integrity may receive official warnings and/or serious academic penalties. Please read more about academic integrity [here](#). If you are unsure about any of these points or feel your assessment might breach academic integrity, please contact your course coordinator for support. It is important that you DO NOT submit any assessment until you can complete the declaration truthfully.

By checking the boxes below, I declare the following:

- ☒ I have not impersonated, or allowed myself to be impersonated by, any person for the purposes of this assessment
- ☒ This assessment is my original work and no part of it has been copied from any other source except where due acknowledgement is made.
- ☒ No part of this assessment has been written for me by any other person except where such collaboration has been authorised by the lecturer/teacher concerned.
- ☒ I have not used generative “AI” tools for the purposes of this assessment.
- ☒ Where this work is being submitted for individual assessment, I declare that it is my original work and that no part has been contributed by, produced by or in conjunction with another student.
- ☒ I give permission for my assessment response to be reproduced, communicated, compared and archived for the purposes of detecting plagiarism.
- ☒ I give permission for a copy of my assessment to be retained by the university for review and comparison, including review by external examiners.

I understand that:

- ☒ Plagiarism is the presentation of the work, idea or creation of another person or machine as though it is your own. It is a form of cheating and is a very serious academic offence that may lead to exclusion from the University. Plagiarised material can be drawn from, and presented in, written, graphic and visual form, including electronic data and oral presentations. Plagiarism occurs when the origin of the material used is not appropriately cited.
- ☒ Plagiarism includes the act of assisting or allowing another person to plagiarise or to copy my work.

I agree and acknowledge that:

- ☒ I have read and understood the Declaration and Statement of Authorship above.

- ☒ If I do not agree to the Declaration and Statement of Authorship in this context and all boxes are not checked, the assessment outcome is not valid for assessment purposes and will not be included in my final result for this course.

Assignment URL

<http://rpubs.com/Prathibha23/1321008>

Scan ID:

[58bd8f8a-a86b-48cf-b1ba-ded6bd7ac3dd](#)

References

[1] Ritchie, H., & Roser, M. (2024). Obesity prevalence among adults (WHO). Our World in Data.

<https://ourworldindata.org/grapher/obesity-prevalence-adults-who-gho>

[2] Ritchie, H., & Roser, M. (2024). Daily caloric supply from carbohydrates, protein and fat. Our World in Data. <https://ourworldindata.org/grapher/daily-caloric-supply-derived-from-carbohydrates-protein-and-fat>

[3] Ritchie, H., & Roser, M. (2024). Dietary composition by country. Our World in Data.

<https://ourworldindata.org/grapher/dietary-composition-by-country>

Libraries used: dplyr, ggplot2, tidyr, plotly readr patchwork

Assignment Code

```
knitr::opts_chunk$set(echo = FALSE, include = FALSE, warning = FALSE, message = FALSE)
```

```
library(dplyr)
```

```
library(ggplot2)
```

```
library(tidyr)
```

```
library(plotly)
```

```
library(readr)
```

```
library(patchwork)
```

```
# Load datasets
```

```
obesity_data <- read_csv("C:/Users/prath/Downloads/obesity-prevalence-adults-who-gho/obesity-prevalence-adults-who-gho.csv")
```

```
diet_data <- read_csv("C:/Users/prath/Downloads/dietary-composition-by-country/dietary-composition-by-country.csv")
```

```
calories_data <- read_csv("C:/Users/prath/Downloads/daily-caloric-supply-derived-from-carbohydrates-protein-and-fat/daily-caloric-supply-derived-from-carbohydrates-protein-and-fat.csv")
```

```
# Step 1: Clean Obesity Data
```

```
obesity_clean <- obesity_data %>%
```

```
  rename(
```

```
    Country = Entity,
```

```
    Obesity_Rate = `Prevalence of obesity among adults, BMI >= 30 (age-standardized estimate) (%) - Sex: both sexes - Age group: 18+ years of age`
```

```
  ) %>%
```

```
  select(Country, Year, Obesity_Rate)
```

```
# Step 2: Clean Diet Composition Data
```

```
diet_clean <- diet_data %>%
```

```
  rename(Country = Entity) %>%
```

```
  select(Country, Year,
```

```
    Sugar_kcal = `Sugar & Sweeteners | 00002909 || Food available for consumption | 0664pc || kilocalories per day per capita`)
```

```
# Step 3: Clean Macronutrient Composition Data
```

```
calories_clean <- calories_data %>%
```

```
  rename(Country = Entity) %>%
```

```
  select(Country, Year,
```

```
    Animal_Protein_kcal = `Daily calorie supply per person that comes from animal protein`,
```

```
    Plant_Protein_kcal = `Daily calorie supply per person that comes from vegetal protein`,
```

```
    Fat_kcal = `Daily calorie supply per person from fat`,
```

```
    Carbs_kcal = `Daily calorie supply per person from carbohydrates`)
```

```
# Step 4: Merge all data
```

```
merged_df <- obesity_clean %>%
```

```

inner_join(diet_clean, by = c("Country", "Year")) %>%

inner_join(calories_clean, by = c("Country", "Year"))


# Step 5: Filter and select clean_df

clean_df <- merged_df %>%

  filter(Year >= 1990) %>%

  select(Country, Year, Obesity_Rate, Sugar_kcal, Fat_kcal, Carbs_kcal, Animal_Protein_kcal,
Plant_Protein_kcal) %>%

  filter(!is.na(Obesity_Rate), !is.na(Sugar_kcal), !is.na(Fat_kcal), !is.na(Carbs_kcal))

# Global Macronutrient Intake Trends (1990–2022)

global_macro <- clean_df %>%

  group_by(Year) %>%

  summarise(

    Sugar = mean(Sugar_kcal),

    Fat = mean(Fat_kcal),

    Carbs = mean(Carbs_kcal),

    Animal_Protein = mean(Animal_Protein_kcal),

    Plant_Protein = mean(Plant_Protein_kcal)

  ) %>%

  pivot_longer(cols = -Year, names_to = "Macronutrient", values_to = "kcal")


macro_order <- c("Sugar", "Plant_Protein", "Animal_Protein", "Fat", "Carbs")

global_macro$Macronutrient <- factor(global_macro$Macronutrient, levels = macro_order)


plot_ly(global_macro,

  x = ~Year, y = ~kcal, color = ~Macronutrient,

  colors = c("Carbs" = "#e07a5f", "Fat" = "#3366CC", "Animal_Protein" = "#9d4edd",

    "Plant_Protein" = "#2E7D32", "Sugar" = "#f6c177"),

  type = 'scatter', mode = 'none', stackgroup = 'one', fill = 'tonexty') %>%

```

```

layout(title = "Global Average Macronutrient Intake (kcal/day) (1990–2022)",

      xaxis = list(title = "Year"),

      yaxis = list(title = "kcal/day"))

# Global Obesity and Nutrition Trends (1990–2022)

global_trend <- clean_df %>%

  group_by(Year) %>%

  summarise(Global_Obesity = mean(Obesity_Rate, na.rm = TRUE))

p1 <- ggplot(global_trend, aes(x = Year, y = Global_Obesity)) +

  geom_line(linewidth = 1) +

  geom_point(color = "blue") +

  labs(title = "Global Average Obesity Rate Over Time",

       x = "Year", y = "Average Obesity Rate (%)") +

  theme_minimal()

ggplotly(p1)

#Macronutrient Correlations

cor_df <- clean_df %>%

  select(Obesity_Rate, Sugar_kcal, Fat_kcal, Carbs_kcal, Animal_Protein_kcal, Plant_Protein_kcal)

cor_matrix <- cor(cor_df, use = "complete.obs")

plot_ly(

  x = colnames(cor_matrix),

  y = rownames(cor_matrix),

  z = cor_matrix,

  type = "heatmap",

  colorscale = "RdBu", zmin = -1, zmax = 1, reversescale = TRUE

) %>%

  layout(title = "Correlation Between Obesity and Macronutrients",

```

```

    xaxis = list(tickangle = 45), yaxis = list(title = ""))

# Obesity vs. Macronutrient Intake

p1 <- ggplot(clean_df, aes(x = Sugar_kcal, y = Obesity_Rate)) +
  geom_point(alpha = 0.3, color = "#1f77b4") +
  geom_smooth(method = "lm", color = "black") +
  labs(title = "Sugar", x = "kcal/day", y = NULL)

p2 <- ggplot(clean_df, aes(x = Fat_kcal, y = Obesity_Rate)) +
  geom_point(alpha = 0.3, color = "#ff7f0e") +
  geom_smooth(method = "lm", color = "black") +
  labs(title = "Fat", x = "kcal/day", y = NULL)

p3 <- ggplot(clean_df, aes(x = Carbs_kcal, y = Obesity_Rate)) +
  geom_point(alpha = 0.3, color = "#2ca02c") +
  geom_smooth(method = "lm", color = "black") +
  labs(title = "Carbohydrates", x = "kcal/day", y = "Obesity Rate (%)")

p4 <- ggplot(clean_df, aes(x = Animal_Protein_kcal, y = Obesity_Rate)) +
  geom_point(alpha = 0.3, color = "#d62728") +
  geom_smooth(method = "lm", color = "black") +
  labs(title = "Animal Protein", x = "kcal/day", y = NULL)

p5 <- ggplot(clean_df, aes(x = Plant_Protein_kcal, y = Obesity_Rate)) +
  geom_point(alpha = 0.3, color = "#9467bd") +
  geom_smooth(method = "lm", color = "black") +
  labs(title = "Plant Protein", x = "kcal/day", y = NULL)

(p1 + p2 + p3) / (p4 + p5) + plot_layout(guides = "collect")

#Top 10 by Average Obesity Rate (1990–2022)

```

```

top10 <- clean_df %>%

  group_by(Country) %>%

  summarise(Avg_Obesity = mean(Obesity_Rate, na.rm = TRUE)) %>%

  arrange(desc(Avg_Obesity)) %>%

  slice(1:10)

p_top10 <- ggplot(top10, aes(x = reorder(Country, Avg_Obesity), y = Avg_Obesity)) +

  geom_col(fill = "#f6c177") +

  coord_flip() +

  labs(title = "Top 10 Countries by Average Obesity Rate", x = "Country", y = "Obesity Rate (%)")

ggplotly(p_top10)

#Predict Obesity From Diet

# Linear Model: Obesity vs Sugar

model_sugar <- lm(Obesity_Rate ~ Sugar_kcal, data = clean_df)

summary(model_sugar)

# Multiple Regression Model

model_multi <- lm(Obesity_Rate ~ Sugar_kcal + Fat_kcal + Carbs_kcal +

  Animal_Protein_kcal + Plant_Protein_kcal, data = clean_df)

summary(model_multi)

# Predicted vs Actual

predicted <- predict(model_multi, clean_df)

p_pred <- ggplot(data.frame(Actual = clean_df$Obesity_Rate, Predicted = predicted),

  aes(x = Predicted, y = Actual)) +

  geom_point(alpha = 0.3, color = "darkblue") +

  geom_abline(slope = 1, intercept = 0, color = "red") +

  labs(title = "Predicted vs Actual Obesity Rate", x = "Predicted", y = "Actual")

ggplotly(p_pred)

```